Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



A Report on Lab-06

Learning Log and Reflection Essay:
Developing a Smart Library System Across the SDLC

Course Code: CSE 3206
Course Title: Software Engineering Sessional

Submitted By

Tanzirul Islam
Roll No.: 2203054
Section: A

Submitted To

Emrana Kabir Hashi
Assistant Professor
Department of Computer Science & Engineering
Rajshahi University of Engineering & Technology

Date: 31 Oct. 2026

1 Learning Log and Reflection Essay

1.1 Technical Decisions

Selecting the **MERN stack** (MongoDB, Express, React, Node.js) was a foundational decision driven by the benefits of a unified JavaScript ecosystem. Having a single programming language across both frontend and backend accelerated development velocity and allowed seamless JSON data exchange without translation layers.

To maintain code modularity, three Gang-of-Four (GoF) design patterns were implemented:

- **Singleton Pattern:** Implemented in `DatabaseManager` to centralize database connection pooling, preventing memory leaks and avoiding redundant connection overhead.
- **Facade Pattern:** The `BorrowFacade` hides the complex multi-step circulation process (member validation, borrowing limits, atomic inventory updates, and audit logging) behind two straightforward methods: `issueBook()` and `returnBook()`.
- **Strategy Pattern:** Instead of hardcoding fragile `if-else` branches, the `FineService` delegates fine calculations to role-specific strategy classes (`StudentFineStrategy` and `FacultyFineStrategy`), adhering to the Open/Closed Principle (OCP).

Furthermore, adopting **Test-Driven Development (TDD)** for the fine engine ensured mathematical correctness. Writing failing assertions for boundary conditions ($d \in \{0, 1, 7, 8\}$) before coding guaranteed that edge-case penalties behaved as intended.

1.2 Engineering Challenges and Solutions

Throughout the implementation, several realistic engineering challenges arose:

- **Branch Collaboration and Merge Conflicts:** Working across separate feature branches led to a merge conflict in `README.md`. Analyzing conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`) and resolving them manually reinforced the importance of clear communication and frequent upstream rebasing.
- **Isolating External Dependencies:** Unit testing the circulation services without mutating live database records was challenging. This was solved by using `jest.fn()` to stub repository calls and employing `mongodb-memory-server` for isolated, fast-running integration tests.
- **Transitioning to a TDD Mindset:** Resisting the natural impulse to code features immediately was difficult. Writing failing tests first felt slower initially, but it significantly reduced debugging time during later refactoring.

1.3 Evaluation of Learning Resources

Navigating various learning materials provided key insights into effective self-directed learning:

- **Official Documentation (Jest & Swagger/OpenAPI):** Official documentation proved indispensable for syntax references and parameter schemas. However, it often lacked guidance on orchestrating tools within complex, multi-tiered architectures.

- **Design Pattern References (Refactoring Guru / GoF):** Conceptual guides provided clear explanations of object-oriented principles. Nonetheless, translating classical class-heavy examples into idiomatic, asynchronous JavaScript required thoughtful adaptation.
- **Learning Insight:** I learned that creating a small, throwaway sandbox project to test a new library before integrating it into production saves substantial debugging hours.

1.4 Plan for the Next Skills to Learn

To further evolve as a software engineer, my upcoming technical goals focus on robust architecture and deployment:

1. **TypeScript Migration:** Transitioning from vanilla JavaScript to TypeScript to gain static compile-time type safety, thereby catching undefined property errors early in development.
2. **Docker and Containerization:** Packaging the backend, database, and client into multi-container Docker environments to eliminate environment discrepancies and solve the classic "it works on my machine" dilemma.
3. **Modern End-to-End Testing:** Deepening my automation testing capabilities by exploring modern E2E testing frameworks such as **Playwright** or **Cypress** for faster, parallel browser automation.